MultiQT notes

November 10, 2026

Notation

These conventions apply to both the methodological notes and the coding updates. Unless stated otherwise, fix one disease, location and forecast horizon; their indices are suppressed.

Forecasts and indices. The indices s, t order forecast reference weeks (`reference_date`, Saturday); j also denotes a historical step inside a re-run. The outcome y_t belongs to the target week of forecast q_t, identified by `target_end_date`, rather than necessarily to its reference week. The quantile levels are $\mathcal{A} = \{\alpha_1 < \cdots < \alpha_m\} \subset (0,1)$, with $m = |\mathcal{A}|$. Write $b_t = (b_t^\alpha)_{\alpha\in\mathcal{A}}$ for the base forecasts and $q_t = (q_t^\alpha)_{\alpha\in\mathcal{A}}$ for the recalibrated forecasts. Unless a transformation is shown explicitly, forecasts and outcomes are in outcome units (hospital counts in the code). The methodological recursions first issue q_t, then observe y_t and update for the next step. The coding section specifies when delayed or revised reports become available instead.

Loss and projections. The indicator is $\mathbf{1}\{\cdot\}$ and $x_+ = \max(x, 0)$. Pinball loss takes the prediction as its first argument and the observation as its second:

$$\rho_\alpha(z, r) = \alpha(r - z)_+ + (1 - \alpha)(z - r)_+. \tag{1}$$

The ordered-quantile cone and Euclidean projection are

$$\mathcal{K} = \{q \in \mathbb{R}^m : q^{\alpha_1} \le \cdots \le q^{\alpha_m}\}, \qquad \Pi_C(x) = \operatorname*{arg\,min}_{z\in C} \|z - x\|^2,$$

where C is nonempty, closed and convex and $\|\cdot\|$ is the Euclidean norm. Assume $b_t \in \mathcal{K}$. For a positive-definite matrix W, write

$$\Pi_{\mathcal{K}}^W(v) = \operatorname*{arg\,min}_{q\in\mathcal{K}} (q - v)^\top W (q - v).$$

Q_p denotes an empirical p-quantile; I denotes the identity matrix.

Scores and tracker states. $S_t(y)$ is a score function, $r_t = S_t(y_t)$ its observed value, and c_t its issued threshold in score units. The target coverage is α, with target miscoverage $\delta = 1 - \alpha$; empirical coverage at a step is $\mathrm{cov}_t^\alpha = \mathbf{1}\{y_t \le q_t^\alpha\}$. The superscripts std and stu distinguish standard and studentized methods. A tilde marks hidden, accumulated state; the untilded state is played after projection. Thus $\tilde\theta_t, \theta_t$ are offsets in outcome units, while $\tilde u_t, u_t$ are thresholds in studentized units. The finite, positive scales are σ_t^α, collected in

$$A_t = \mathrm{diag}(\sigma_t^{\alpha_1}, \ldots, \sigma_t^{\alpha_m});$$

σ_t denotes a scale common to all levels and σ a constant scale. The tracker step size is η (or η_t when varying), and κ is a rate multiplier. The selected loss subgradient is written g_α; its form is derived in (8).

Reports, feedback and re-runs. The date ι_t is the information cutoff for forecast q_t: normally the Wednesday of its reference week, with actual generation dates used where known. The notation $y_s^{[v]}$ means the reported count for the target week of q_s in the latest report dated on or before date v (the archive's `version` field). A valid report is finite and nonnegative. Its first valid publication date is ν_s, and $d \geq 0$ is an additional delay in calendar weeks. The symbols $m_s^{(d)}$, $\tau_d(s)$ and $\mathcal{R}_t^{(d)}$ denote the maturity date, the index of the round that consumes feedback for q_s, and the set of rows consumed before issuing q_t, respectively; their scheduling rules are given in (28)–(29). The set $I_{20}^{(d)}$ contains the first 20 consumed forecast rows in release order, with their stateful report values

$$\bar{y}_s^{(d)} = y_s^{[\iota_{\tau_d(s)}]}, \qquad s \in I_{20}^{(d)}. \tag{2}$$

For forecasts and states, a superscript $[t]$ instead identifies the re-run at reference-week index t: $q_s^{[t]}$ is the forecast for row s reconstructed inside that re-run. Thus v in $y_s^{[v]}$ is a report-cutoff date, while t in $q_s^{[t]}$ is a re-run index. Feedback respects the cutoff convention, subject to the retrospective initialization described in Section 2.1.2.

Implementation scales and expert weights. The tracker transformation is $\psi(x) = x$ or $\psi(x) = \log(1 + x)$. The count divisor $c_{\text{scale}} > 0$ is population divided by 100,000 by default, so y/c_{scale} is a rate per 100,000 residents. The fixed reference constants S_0 (initial error scale) and L_0 (expert loss normalization) are defined in (34) and (35); S_0 is separate from the time-indexed score function S_t. Their numerical floors are ε_S and ε_L. For one expert-weight update, suppressing the time index, f_k^α is expert k's forecast, w_k its mixture weight, and λ the weight-learning rate. The normalized losses are ℓ_k^α for one level and ℓ_k for their mean across levels.

1 Methodological notes

1.1 Studentized score

Writing the score as a signed residual gives the usual additive MultiQT correction. Studentizing that residual gives multiplicatively scaled offsets, or, equivalently, offsets tracked in scaled coordinates. A changing scale rescales the accumulated correction as well as each new increment. If the scale decreases after a peak, it can shrink the outcome-scale contribution of a positive correction accumulated during an upswing. This is the motivation; the hidden accumulated state is not erased.

With quantile-specific scales, the ordering constraint defines a diagonally transformed, shifted isotonic cone. This is generally not a cone itself, but it remains a nonempty closed convex polyhedron with a unique Euclidean projection, computable through weighted isotonic regression in forecast coordinates. The gradient steps use a weighted quantile loss. The coverage criterion and calibration target are unchanged (strictly increasing invertible transformation); calibration is not guaranteed.

1.1.1 Standard MultiQT

The original MultiQT procedure [1] is

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t^{\text{std}} = b_t + \theta_t, \tag{3}$$

$$\text{cov}_t^{\text{std},\alpha} = \mathbf{1}\{y_t \le q_t^{\text{std},\alpha}\}, \qquad \tilde{\theta}_{t+1}^{\alpha} = \tilde{\theta}_t^{\alpha} - \eta(\text{cov}_t^{\text{std},\alpha} - \alpha). \tag{4}$$

1.1.2 Score-threshold tracking

To make the connection explicit, first consider the scalar score-threshold update of Angelopoulos et al. [2]. We then recover the MultiQT update using the signed residual as the score.

The score and threshold determine the prediction set

$$\mathcal{P}_t = \{y : S_t(y) \le c_t\}, \qquad \text{err}_t = \mathbf{1}\{r_t > c_t\}. \tag{5}$$

Update according to

$$\begin{aligned} c_{t+1} &= c_t + \eta(\text{err}_t - \delta) \\ &= c_t - \eta(\mathbf{1}\{r_t \le c_t\} - \alpha). \end{aligned} \tag{6}$$

The second line follows from $\text{err}_t = 1 - \mathbf{1}\{r_t \le c_t\}$ and $\delta = 1 - \alpha$. The prediction set is obtained by score inversion. For the signed affine scores below, that set's endpoint is the outcome quantile forecast.

Holding the observed score r fixed, the subdifferential of (1) in the threshold is

$$\partial_z \rho_\alpha(z, r) = \begin{cases} \{-\alpha\}, & z < r, \\ [-\alpha, 1-\alpha], & z = r, \\ \{1-\alpha\}, & z > r. \end{cases} \tag{7}$$

Selecting $1 - \alpha$ at equality gives

$$g_\alpha(z; r) = \mathbf{1}\{r \le z\} - \alpha \ \in \ \partial_z \rho_\alpha(z, r). \tag{8}$$

Thus $c_{t+1} = c_t - \eta g_\alpha(c_t; r_t)$ is (6). The score's distance from the threshold affects the loss; the side of the threshold determines the update direction and size.

1.1.3 MultiQT in residual coordinates

For each level, define the score and its observed value by

$$S_t^{\mathrm{std},\alpha}(y) = y - b_t^\alpha, \qquad r_t^{\mathrm{std},\alpha} = y_t - b_t^\alpha. \tag{9}$$

Using θ_t^α as the score threshold gives

$$\{y : S_t^{\mathrm{std},\alpha}(y) \le \theta_t^\alpha\} = (-\infty, b_t^\alpha + \theta_t^\alpha] = (-\infty, q_t^{\mathrm{std},\alpha}]. \tag{10}$$

Consequently,

$$\mathrm{cov}_t^{\mathrm{std},\alpha} = \mathbf{1}\{y_t \le b_t^\alpha + \theta_t^\alpha\} = \mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\}. \tag{11}$$

Substitution into (4) gives

$$\boxed{\tilde{\theta}_{t+1}^\alpha = \tilde{\theta}_t^\alpha - \eta(\mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\} - \alpha).} \tag{12}$$

The original MultiQT loss has the exact residual-coordinate representation

$$\begin{aligned} \ell_t^{\mathrm{std}}(\theta) &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(b_t^\alpha + \theta^\alpha, y_t) \\ &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(\theta^\alpha, y_t - b_t^\alpha) = \sum_{\alpha\in\mathcal{A}} \rho_\alpha(\theta^\alpha, r_t^{\mathrm{std},\alpha}). \end{aligned} \tag{13}$$

The threshold c_t of the scalar score tracker corresponds to the offset θ_t^α when the score is $y - b_t^\alpha$. For a single level the constraint is all of $\mathbb{R}$, so the hidden and played states coincide and (12) is exactly scalar quantile tracking. For multiple levels, MultiQT uses a joint projection: evaluate g_α at the played θ_t^α, but apply the increment to the hidden $\tilde{\theta}_t^\alpha$.

1.1.4 Studentized residuals and their update

Now replace the residual score by the studentized residual.

For this construction, choose the scales from information available at issuance and hold them fixed when optimizing the threshold. Define

$$S_t^{\mathrm{stu},\alpha}(y) = \frac{y - b_t^\alpha}{\sigma_t^\alpha}, \qquad r_t^{\mathrm{stu},\alpha} = \frac{y_t - b_t^\alpha}{\sigma_t^\alpha}. \tag{14}$$

Inverting the score gives

$$\begin{aligned} S_t^{\mathrm{stu},\alpha}(y) \le u_t^\alpha \quad &\Longleftrightarrow \quad y \le b_t^\alpha + \sigma_t^\alpha u_t^\alpha, \\ q_t^{\mathrm{stu},\alpha} = b_t^\alpha + \sigma_t^\alpha u_t^\alpha, &\qquad \mathrm{cov}_t^{\mathrm{stu},\alpha} = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\}. \end{aligned} \tag{15}$$

Standard MultiQT tracks the outcome-unit residual threshold θ_t^α and issues $b_t^\alpha + \theta_t^\alpha$. Studentization tracks u_t^α and issues $b_t^\alpha + \sigma_t^\alpha u_t^\alpha$. The factor σ_t^α enters through the inverse score map.

Apply the same quantile-loss construction to the studentized scores:

$$\begin{aligned} \ell_t^{\mathrm{stu}}(u) &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(u^\alpha, r_t^{\mathrm{stu},\alpha}) \\ &= \sum_{\alpha\in\mathcal{A}} \frac{1}{\sigma_t^\alpha} \rho_\alpha(b_t^\alpha + \sigma_t^\alpha u^\alpha, y_t). \end{aligned} \tag{16}$$

Using (8), its selected subgradient is

$$g_t^{\mathrm{stu},\alpha}(u_t) = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\} - \alpha = \mathrm{cov}_t^{\mathrm{stu},\alpha} - \alpha. \tag{17}$$

The chain rule gives the same result. Away from ties, for one level,

$$\frac{\partial}{\partial u^\alpha}\left[\frac{\rho_\alpha(b_t^\alpha+\sigma_t^\alpha u^\alpha, y_t)}{\sigma_t^\alpha}\right] = \frac{1}{\sigma_t^\alpha}(\mathbf{1}\{y_t \le b_t^\alpha+\sigma_t^\alpha u^\alpha\}-\alpha)\sigma_t^\alpha$$
$$= \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u^\alpha\}-\alpha. \tag{18}$$

At a tie use the selected subgradient in (8). The inverse scale in the loss cancels the scale in the inverse score map.

Retaining MultiQT's lazy update therefore gives

$$\boxed{\tilde{u}_{t+1}^\alpha = \tilde{u}_t^\alpha - \eta g_t^{\mathrm{stu},\alpha}(u_t) = \tilde{u}_t^\alpha - \eta\left(\mathbf{1}\left\{\frac{y_t-b_t^\alpha}{\sigma_t^\alpha} \le u_t^\alpha\right\}-\alpha\right).} \tag{19}$$

Initialize $\tilde{u}_1 = 0$. The score is compared with the *played* threshold; the increment is applied to the *hidden* threshold. The projection that produces u_t is specified next.

	Standard MultiQT	Studentized extension
Observed score	$r_t^{\mathrm{std},\alpha} = y_t - b_t^\alpha$	$r_t^{\mathrm{stu},\alpha} = (y_t - b_t^\alpha)/\sigma_t^\alpha$
Gradient	$\mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\}-\alpha$	$\mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\}-\alpha$
Update origin	$\tilde{\theta}_t^\alpha$	$\tilde{u}_t^\alpha$
Issued forecast	$b_t^\alpha + \theta_t^\alpha$	$b_t^\alpha + \sigma_t^\alpha u_t^\alpha$

The gradient has the same form because both losses apply quantile loss to a score threshold. The observed scores, state units, and issued forecasts change. An unnormalized outcome-unit loss would instead have gradient $\sigma_t^\alpha(\mathrm{cov}_t^{\mathrm{stu},\alpha} - \alpha)$ with respect to u^α, and would define a different update.

1.1.5 Ordering constraints and projection

The constraint always concerns the outcome forecasts: $q_t^{\alpha_1} \le \cdots \le q_t^{\alpha_m}$. For the ordinary residual score, this requires

$$b_t + \theta \in \mathcal{K} \iff \theta \in C_t^{\mathrm{std}} := \mathcal{K} - b_t, \tag{20}$$
$$\theta^{\alpha_i} - \theta^{\alpha_{i+1}} \le b_t^{\alpha_{i+1}} - b_t^{\alpha_i}, \qquad i = 1,\dots,m-1. \tag{21}$$

For studentization, $q_t^{\mathrm{stu}} = b_t + A_t u_t$, so

$$\boxed{C_t^{\mathrm{stu}} = \{u \in \mathbb{R}^m : b_t + A_t u \in \mathcal{K}\} = A_t^{-1}(\mathcal{K} - b_t).} \tag{22}$$

This is a closed convex polyhedron given by linear inequalities. It contains zero when $b_t \in \mathcal{K}$. The shift means that, like $\mathcal{K} - b_t$, it is generally not a cone. For unequal scales, $A_t^{-1}\mathcal{K} \neq \mathcal{K}$ in general, so the set cannot be simplified to $\mathcal{K} - A_t^{-1}b_t$. The standard constraint $b_t + \theta \in \mathcal{K}$ is replaced by $b_t + A_t u \in \mathcal{K}$, and the Euclidean projection is well defined:

$$u_t = \Pi_{C_t^{\mathrm{stu}}}(\tilde{u}_t), \qquad q_t^{\mathrm{stu}} = b_t + A_t u_t. \tag{23}$$

Substituting $u = A_t^{-1}(q - b_t)$ into the projection objective yields

$$q_t^{\mathrm{stu}} = \arg\min_{q\in\mathcal{K}} \|A_t^{-1}(q-b_t) - \tilde{u}_t\|^2$$
$$= \arg\min_{q\in\mathcal{K}} \sum_{\alpha\in\mathcal{A}} \frac{(q^\alpha - b_t^\alpha - \sigma_t^\alpha \tilde{u}_t^\alpha)^2}{(\sigma_t^\alpha)^2}. \tag{24}$$

In weighted-projection notation, the exact identity is

$$\boxed{b_t + A_t \Pi_{A_t^{-1}(\mathcal{K}-b_t)}(\tilde{u}_t) = \Pi_{\mathcal{K}}^{A_t^{-2}}(b_t + A_t \tilde{u}_t).} \tag{25}$$

Thus the implementation is weighted isotonic regression of $v_t^\alpha = b_t^\alpha + \sigma_t^\alpha \tilde{u}_t^\alpha$ with weights $1/(\sigma_t^\alpha)^2$.

The ordinary residual case has forecast-space metric I; the studentized case has metric A_t^{-2}. These weights follow from the chosen Euclidean geometry of the tracked score coordinates.

1.1.6 Common and constant scales

For $S_t^\alpha(y) = (y - b_t^\alpha)/\sigma_t$, we have $A_t = \sigma_t I$. Positive scalar multiplication preserves $\mathcal{K}$:

$$C_t^{\text{stu}} = \mathcal{K} - b_t/\sigma_t, \qquad q_t^{\text{stu}} = \Pi_{\mathcal{K}}(b_t + \sigma_t \tilde{u}_t). \tag{26}$$

All projection weights are equal and cancel. Equivalently, run standard MultiQT on base b_t/σ_t and outcome y_t/σ_t, then multiply the issued forecast by σ_t.

If $\sigma_t = \sigma$ is constant, standard MultiQT on the original inputs is also equivalent after converting the state and learning rate:

$$\tilde{\theta}_t = \sigma \tilde{u}_t, \qquad \eta_{\text{std}} = \sigma \eta_{\text{stu}}. \tag{27}$$

1.2 Questions and ideas

- What other scores would be reasonable to consider, i.e. what property of the base forecast do I need to recalibrate? Instead of asking for 1d PIT calibration, could I ask for 1d hdr-calibration? With studentized scores, or in general strictly increasing and invertible scores, the coverage indicator and calibration target do not change

2 Coding Updates

2.1 9 November 2026

2.1.1 Feedback handling: Stateful, re-run and delay

The feedback design is independent of the recalibration method and uses the report and cutoff conventions in Notation.

Stateful (learn once). Wait d calendar weeks from the first valid report. At the next forecast cutoff ι_t with $t > s$, look up the latest report. If it is invalid, wait for a later forecast; never substitute an older report.

$$m_s^{(d)} = \nu_s + 7d \text{ days}, \tag{28}$$

$$\tau_d(s) = \min\{t > s : \iota_t \geq m_s^{(d)},\ y_s^{[\iota_t]} \text{ is valid}\}, \qquad \mathcal{R}_t^{(d)} = \{s : \tau_d(s) = t\}. \tag{29}$$

An empty $\mathcal{R}_t^{(d)}$ means no update. For each released row, use $y_s^{[\iota_t]}$. If a missing forecast postpones the update so that $\iota_t > m_s^{(d)}$, select the report at ι_t, rather than fixing it at $m_s^{(d)}$. Feedback for each q_s is used at most once; revisions after that update are ignored.

For plain MultiQT on counts, start at $\tilde{\theta}_0 = 0$; when learning is active, update using previously issued q_s^α, then issue q_t:

$$\tilde{\theta}_t^\alpha = \tilde{\theta}_{t-1}^\alpha - \eta_t \sum_{s \in \mathcal{R}_t^{(d)}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^\alpha\} - \alpha), \tag{30}$$

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t = b_t + \theta_t. \tag{31}$$

Sum with η_t; keep the hidden state unprojected. For log tracking, apply the same update in ψ coordinates. The default retrospective initialization of S_0 is described in Section 2.1.2.

Weekly re-run. At each reference-week index t, re-run MultiQT from the start using the latest reports $y_s^{[\iota_t]}$. Only the new forecast $q_t^{[t]}$ is issued; previously issued forecasts remain unchanged. Starting from $\tilde{\theta}_0^{[t]} = 0$, replay $j = 1, \ldots, t$ with feedback only at $j = \tau_d(s)$, retaining delay d; omit invalid reports:

$$\tilde{\theta}_j^{\alpha,[t]} = \tilde{\theta}_{j-1}^{\alpha,[t]} - \eta_j^{[t]} \sum_{\substack{s \in \mathcal{R}_j^{(d)} \\ y_s^{[\iota_t]} \text{ valid}}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^{\alpha,[t]}\} - \alpha), \tag{32}$$

$$\theta_j^{[t]} = \Pi_{\mathcal{K}-b_j}(\tilde{\theta}_j^{[t]}), \qquad q_j^{[t]} = b_j + \theta_j^{[t]}. \tag{33}$$

For each case and feedback delay, estimate S_0 and the reference expert loss L_0 once from the first 20 feedback rows consumed by the stateful run. Every weekly re-run reuses these constants. With warmup replay enabled, the constants are applied before the 20th feedback row is released, making initialization retrospective; see Section 2.1.2 for the formulas and timing.

Studentized methods use $\tilde{u}$, q^{stu} and projection (23), issuing the base until their scale is available; re-runs reconstruct the scale history. Evaluation uses a common latest-report truth, separate from learning.

2.1.2 Tuning, initialization and loss normalization

The reference scales are estimated from $I_{20}^{(d)}$ and its stateful feedback values $\bar{y}_s^{(d)}$; these 20 rows need not span 20 calendar weeks. Method parameters, such as a rate multiplier or an expert-weighting lookback, are selected on a tuning period. These are distinct steps in the implementation.

Initial error scale S_0. Pool absolute base errors across the reference rows and all quantile levels, and take their empirical 90th percentile:

$$S_0 = \max\Big\{\varepsilon_S, Q_{0.9}\Big(\Big\{|\psi(b_s^\alpha) - \psi(\bar{y}_s^{(d)})| : s \in I_{20}^{(d)},\ \alpha \in \mathcal{A}\Big\}\Big)\Big\}. \tag{34}$$

Here $\varepsilon_S = 10^{-6}$ is a numerical floor. The fixed-rate tracker uses $\eta = \kappa S_0$, measured in tracker units. The studentized rate is instead chosen directly in standardized units. In the code, S_0 also sets the studentized-scale bounds $[0.05S_0, S_0]$.

Expert loss normalization (new). Using the pinball loss in (1), define one reference loss per case and delay:

$$L_0 = \max\left\{\varepsilon_L, \frac{1}{20|\mathcal{A}|}\sum_{s\in I_{20}^{(d)}}\sum_{\alpha\in\mathcal{A}}\rho_\alpha\left(\frac{b_s^\alpha}{c_{\text{scale}}}, \frac{\bar{y}_s^{(d)}}{c_{\text{scale}}}\right)\right\}, \qquad \varepsilon_L = 10^{-6}. \tag{35}$$

The losses supplied to exponential weighting are

$$\ell_k^\alpha = \frac{\rho_\alpha(f_k^\alpha/c_{\text{scale}}, y/c_{\text{scale}})}{L_0}, \qquad \ell_k = \frac{1}{|\mathcal{A}|}\sum_{\alpha\in\mathcal{A}}\ell_k^\alpha. \tag{36}$$

Per-level weighting uses ℓ_k^α; common weighting uses ℓ_k. Both divide by the same scalar L_0. For example, the common-weight exponential step, before optional sharing, is

$$w_k^{\text{new}} = \frac{w_k \exp(-\lambda\ell_k)}{\sum_j w_j \exp(-\lambda\ell_j)}. \tag{37}$$

Previously, expert weighting used the unnormalized pinball losses. The new division makes λ unit-free: changing c_{scale} rescales the numerator and denominator equally, provided the numerical floor is inactive. These weighting losses remain on the natural count scale even for a log tracker. During a re-run, the numerator uses the current report and the reconstructed expert forecast, while L_0 stays at its stateful value.

Retrospective initialization. The default `replay_warmup = TRUE` computes both constants from the eventual first 20 releases, then applies them from the start. Forecasts before the 20th release can therefore depend on information unavailable at their issue time through these constants. This initialization is retrospective even though feedback updates follow the schedule above. With replay disabled, learning waits until the warmup is complete; if fewer than 20 rows are released, learning remains inactive. Once estimated, S_0 and L_0 remain fixed across weekly re-runs. Corrections, expert weights and time-varying studentized scales are still reconstructed in each re-run. The reference scales are computed by `error_scale()` and `weighting_loss_scale()` in `R/03_functions.R`.

Parameter tuning. `R/05_tuning.R` selects parameters separately for each disease, tracker and feedback design (delay 0, delay 1 and weekly re-run). For the active hub design, tuning uses target weeks through 31 May 2025 and evaluation starts in June 2025. Each candidate is scored against the latest-report truth, averaging pinball loss over tuning weeks and quantile levels within each location–horizon case, then equally over the valid cases. All prepared locations and horizons $-1, \ldots, 3$ enter the hub design's tuning. Count trackers minimize loss per 100,000 residents; log trackers minimize loss of $\log(1 + y)$. This tuning criterion is not divided by L_0, and calibration error is reported but not optimized. The smallest-loss grid value is selected, with the smaller value at ties. Soft-switch settings are selected sequentially, followed by the outer correction rates.

2.1.3 Effect of increasing feedback delay

2.1.4 US and the five largest states

2.1.5 Extended data history

2.1.6 Trade-off and evaluation plots, including forecast curves

2.1.7 Studentized scores and non-additive updates

2.1.8 A small set of phase-specific experts

2.1.9 Learned weights shared across quantile levels

References

[1] Tiffany Ding, Isaac Gibbs, and Ryan J. Tibshirani. *Calibrated Multi-Level Quantile Forecasting.* arXiv:2512.23671v1, 2025. MultiQT paper.

[2] Anastasios N. Angelopoulos, Emmanuel J. Candès, and Ryan J. Tibshirani. *Conformal PID Control for Time Series Prediction.* Advances in Neural Information Processing Systems 36, 2023. NeurIPS paper.

[3] CMU Delphi. *Delphi hub forecasting: calibration experiments and tracker code.* Pinned source revision `9abebcfb`, inspected 9 November 2026. E09: constant learning rate and revision delay; tracker implementation.